

# **LAPORAN PRAKTIKUM TRAINING MODEL CUSTOM CNN PADA DATASET CIFAR-10**

Dosen Pengampu:

Bayu Adhi Nugroho, Ph.D



**UIN SUNAN AMPEL  
S U R A B A Y A**

Nama : Afrizal Dwi Ahmad

NIM : 09010624001

**SISTEM INFORMASI**

**SAINS DAN TEKNOLOGI**

**UNIVERSITAS ISLAM NEGERI SUNAN AMPEL SURABAYA**

**2026**

## Spesifikasi perangkat yang digunakan:

- OS: Kubuntu 26.04.1 LTS (Resolute Raccoon) x86\_64
- CPU: 12th Gen Intel(R) Core(TM) i5-12450HX (12) @ 4.40 GHz
- GPU 1: NVIDIA GeForce RTX 3050 6GB Laptop GPU [Discrete]
- GPU 2: Intel UHD Graphics @ 1.30 GHz [Integrated]
- Memory: 32 GiB

## 1. Import Library

Langkah pertama adalah mengimpor seluruh library yang diperlukan. Library utama yang digunakan meliputi TensorFlow/Keras untuk membangun dan melatih model deep learning, scikit-learn untuk splitting data dan evaluasi, matplotlib untuk visualisasi, serta SciKeras (KerasClassifier) sebagai wrapper agar model Keras dapat digunakan dengan GridSearchCV dari scikit-learn.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
%matplotlib inline

import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import load_model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Conv2D, MaxPool2D, Flatten, Dropout, BatchNormalization

from scikeras.wrappers import KerasClassifier

from sklearn.model_selection import GridSearchCV, KFold
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
```

## Penjelasan library yang digunakan:

- pandas & numpy: manipulasi data dan komputasi numerik.
- matplotlib: visualisasi grafik dan plot.
- tensorflow.keras: framework deep learning untuk membangun model CNN.
- ImageDataGenerator: augmentasi data untuk meningkatkan generalisasi model.
- EarlyStopping: callback untuk menghentikan training jika model sudah konvergen.

- KerasClassifier (SciKeras): wrapper agar model Keras kompatibel dengan GridSearchCV sklearn.
- GridSearchCV & KFold: pencarian hyperparameter terbaik dengan cross-validation.
- classification\_report & confusion\_matrix: metrik evaluasi klasifikasi.

## 2. Load Data

Dataset CIFAR-10 dimuat menggunakan fungsi bawaan Keras. Hanya data training (50.000 gambar) yang dimuat pada tahap ini karena data test akan digunakan secara terpisah di notebook testing. Setiap gambar memiliki dimensi 32×32 piksel dengan 3 channel warna (RGB).

```
(X_train, y_train), _ = cifar10.load_data()
```

```
print(f"X_train shape: {X_train.shape}")
```

```
print(f"y_train shape: {y_train.shape}")
```

```
X_train shape: (50000, 32, 32, 3)
```

```
y_train shape: (50000, 1)
```

Output menunjukkan bahwa terdapat 50.000 gambar dengan dimensi 32×32×3 (RGB) dan label berupa array 1 dimensi.

```
# Define the labels of the dataset
```

```
labels = ['airplane', 'automobile', 'bird', 'cat', 'deer',  
         'dog', 'frog', 'horse', 'ship', 'truck']
```

Dataset memiliki 10 kelas, yaitu: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, dan truck.

## 3. Visualisasi Distribusi Kelas

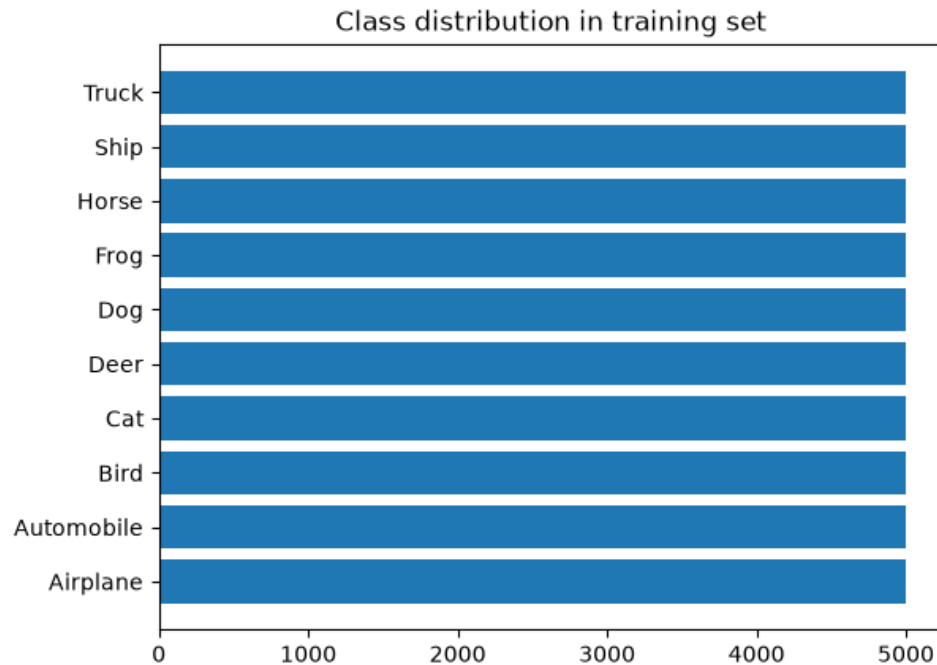
Distribusi kelas pada data training divisualisasikan menggunakan bar chart horizontal untuk memastikan bahwa data terdistribusi secara seimbang (balanced).

```
classes_name = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer', 'Dog', 'Frog', 'Horse', 'Ship', 'Truck']
```

```
classes, counts = np.unique(y_train, return_counts=True)
```

```
plt.barh(classes_name, counts)
```

```
plt.title('Class distribution in training set')
```



*Gambar 1. Distribusi kelas pada data training CIFAR-10*

Dari visualisasi terlihat bahwa setiap kelas memiliki jumlah sampel yang sama, yaitu masing-masing 5.000 gambar. Hal ini menunjukkan bahwa dataset CIFAR-10 bersifat balanced, sehingga tidak diperlukan teknik resampling atau class weighting.

## 4. Data Preprocessing

Tahap preprocessing meliputi normalisasi piksel, splitting data, dan one-hot encoding label.

```
# Scale the data
X_train = X_train / 255.0

# Split training data into train (80%) and validation (20%)
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train,
    test_size=0.2,
    random_state=23092026,
    stratify=y_train
)

# Transform target variable into one-hotencoding
y_cat_train = to_categorical(y_train, 10)
y_cat_val = to_categorical(y_val, 10)

print(f'X_train shape: {X_train.shape}, y_cat_train shape: {y_cat_train.shape}')
```

```
print(f'X_val shape : {X_val.shape}, y_cat_val shape : {y_cat_val.shape}')
```

```
X_train shape: (40000, 32, 32, 3), y_cat_train shape: (40000, 10)
```

```
X_val shape : (10000, 32, 32, 3), y_cat_val shape : (10000, 10)
```

### Penjelasan tahapan preprocessing:

- Normalisasi: Nilai piksel dibagi 255.0 agar berada dalam rentang [0, 1]. Hal ini mempercepat konvergensi dan menstabilkan proses training.
- Train-Validation Split: Data dibagi 80:20 menggunakan `train_test_split` dengan `random_state=23092026` untuk reproduktibilitas. Parameter `stratify=y_train` digunakan agar proporsi kelas tetap seimbang di kedua subset.
- One-Hot Encoding: Label kategori dikonversi menjadi representasi vektor biner menggunakan `to_categorical`. Misalnya, kelas 3 (cat) menjadi [0,0,0,1,0,0,0,0,0,0].

Setelah preprocessing, data training terdiri dari 40.000 gambar dan data validasi terdiri dari 10.000 gambar.

## 5. Arsitektur Model Custom CNN

Model Custom CNN dibangun menggunakan arsitektur Sequential dari Keras. Model terdiri dari 3 blok konvolusional dan 1 classification head. Fungsi `build_model` menerima parameter `dense_units` dan `dropout_rate` sebagai hyperparameter yang akan dituning.

```
INPUT_SHAPE = (32, 32, 3)
```

```
KERNEL_SIZE = (3, 3)
```

```
def build_model(dense_units=128, dropout_rate=0.25):
```

```
    model = Sequential([
```

```
        # Top-level Input layer
```

```
        Input(shape=INPUT_SHAPE),
```

```
        # Block 1
```

```
        Conv2D(filters=32, kernel_size=KERNEL_SIZE, activation='relu', padding='same'),
```

```
        BatchNormalization(),
```

```
        Conv2D(filters=32, kernel_size=KERNEL_SIZE, activation='relu', padding='same'),
```

```
        BatchNormalization(),
```

```
        MaxPool2D(pool_size=(2, 2)),
```

```
        Dropout(dropout_rate),
```

```
        # Block 2
```

```
        Conv2D(filters=64, kernel_size=KERNEL_SIZE, activation='relu', padding='same'),
```

```
        BatchNormalization(),
```

```
        Conv2D(filters=64, kernel_size=KERNEL_SIZE, activation='relu', padding='same'),
```

```
        BatchNormalization(),
```

```
        MaxPool2D(pool_size=(2, 2)),
```

```

Dropout(dropout_rate),

# Block 3
Conv2D(filters=128, kernel_size=KERNEL_SIZE, activation='relu', padding='same'),
BatchNormalization(),
Conv2D(filters=128, kernel_size=KERNEL_SIZE, activation='relu', padding='same'),
BatchNormalization(),
MaxPool2D(pool_size=(2, 2)),
Dropout(dropout_rate),

# Classification Head
Flatten(),
Dense(dense_units, activation='relu'),
Dropout(dropout_rate),
Dense(10, activation='softmax')
])

METRICS = [
    'accuracy',
    tf.keras.metrics.Precision(name='precision'),
    tf.keras.metrics.Recall(name='recall')
]
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=METRICS)
return model

```

### Penjelasan arsitektur model:

- Block 1 (filters=32): Dua layer Conv2D dengan 32 filter (3×3), aktivasi ReLU, padding same, diikuti BatchNormalization, MaxPooling (2×2), dan Dropout.
- Block 2 (filters=64): Dua layer Conv2D dengan 64 filter (3×3), konfigurasi serupa dengan Block 1.
- Block 3 (filters=128): Dua layer Conv2D dengan 128 filter (3×3), konfigurasi serupa dengan Block 1.
- Classification Head: Layer Flatten untuk mengubah feature map menjadi vektor 1D, Dense layer dengan jumlah unit yang dapat dituning, Dropout, dan Dense output layer (10 neuron, softmax) untuk klasifikasi 10 kelas.
- Loss Function: categorical\_crossentropy karena ini adalah klasifikasi multi-kelas dengan one-hot encoding.
- Optimizer: Adam (Adaptive Moment Estimation) yang menggabungkan momentum dan RMSProp.
- Metrics: Accuracy, Precision, dan Recall untuk evaluasi komprehensif.

### Ringkasan arsitektur model setelah di-compile:

| Layer (type) | Output Shape | Param # |  |
|--------------|--------------|---------|--|
|              |              |         |  |

|                                |                    |         |
|--------------------------------|--------------------|---------|
| conv2d (Conv2D)                | (None, 32, 32, 32) | 896     |
| batch_normalization (BN)       | (None, 32, 32, 32) | 128     |
| conv2d_1 (Conv2D)              | (None, 32, 32, 32) | 9,248   |
| batch_normalization_1 (BN)     | (None, 32, 32, 32) | 128     |
| max_pooling2d (MaxPooling2D)   | (None, 16, 16, 32) | 0       |
| dropout (Dropout)              | (None, 16, 16, 32) | 0       |
| conv2d_2 (Conv2D)              | (None, 16, 16, 64) | 18,496  |
| batch_normalization_2 (BN)     | (None, 16, 16, 64) | 256     |
| conv2d_3 (Conv2D)              | (None, 16, 16, 64) | 36,928  |
| batch_normalization_3 (BN)     | (None, 16, 16, 64) | 256     |
| max_pooling2d_1 (MaxPooling2D) | (None, 8, 8, 64)   | 0       |
| dropout_1 (Dropout)            | (None, 8, 8, 64)   | 0       |
| conv2d_4 (Conv2D)              | (None, 8, 8, 128)  | 73,856  |
| batch_normalization_4 (BN)     | (None, 8, 8, 128)  | 512     |
| conv2d_5 (Conv2D)              | (None, 8, 8, 128)  | 147,584 |
| batch_normalization_5 (BN)     | (None, 8, 8, 128)  | 512     |
| max_pooling2d_2 (MaxPooling2D) | (None, 4, 4, 128)  | 0       |
| dropout_2 (Dropout)            | (None, 4, 4, 128)  | 0       |
| flatten (Flatten)              | (None, 2048)       | 0       |
| dense (Dense)                  | (None, 128)        | 262,272 |
| dropout_3 (Dropout)            | (None, 128)        | 0       |
| dense_1 (Dense)                | (None, 10)         | 1,290   |

Total params: 552,362 (2.11 MB)

Trainable params: 551,466 (2.10 MB)

Non-trainable params: 896 (3.50 KB)

Model memiliki total 552.362 parameter, dengan 551.466 parameter yang dapat dilatih (trainable) dan 896 parameter non-trainable (berasal dari BatchNormalization).

## 6. Hyperparameter Tuning dengan GridSearchCV

### Training Model Baseline (Sebelum GridSearch)

Sebelum menjalankan GridSearchCV, terlebih dahulu dilatih model baseline dengan hyperparameter default (dense\_units=128, dropout\_rate=0.25) selama 10 epoch menggunakan data augmentation. Tujuannya adalah untuk mendapatkan baseline performa yang akan dibandingkan dengan hasil setelah tuning.

```
# 1. Evaluate Baseline Model (Before GridSearch: dense_units=128, dropout_rate=0.25)
print("--- 1. Evaluating Baseline Model (Before GridSearch) ---")
baseline_model = build_model(dense_units=128, dropout_rate=0.25)

batch_size = 32
baseline_datagen = ImageDataGenerator(width_shift_range=0.1, height_shift_range=0.1, horizontal_flip=True)
baseline_train_gen = baseline_datagen.flow(X_train, y_cat_train, batch_size=batch_size)
steps_per_epoch = X_train.shape[0] // batch_size

baseline_history = baseline_model.fit(
    baseline_train_gen,
    epochs=10,
    steps_per_epoch=steps_per_epoch,
```

```

validation_data=(X_val, y_cat_val),
verbose=1
)

baseline_eval = baseline_model.evaluate(X_val, y_cat_val, verbose=0)
baseline_val_loss = baseline_eval[0]
baseline_val_acc = baseline_eval[1]
print(f"\nBaseline (Before GridSearch) -> Val Loss: {baseline_val_loss:.4f}, Val Accuracy: {baseline_val_acc * 100:.2f}%")

```

--- 1. Evaluating Baseline Model (Before GridSearch) ---

```

Epoch 1/10
1250/1250 ————— 30s 13ms/step - accuracy: 0.3822 - loss: 1.7079 - ...
Epoch 2/10
1250/1250 ————— 12s 10ms/step - accuracy: 0.5295 - loss: 1.3123 - ...
...
Epoch 10/10
1250/1250 ————— 13s 10ms/step - accuracy: 0.7621 - loss: 0.7019 - val_accuracy: 0.7730 - val_loss:
0.6602

```

Baseline (Before GridSearch) -> Val Loss: 0.6602, Val Accuracy: 77.30%

Hasil baseline menunjukkan model mencapai Validation Accuracy sebesar 77.30% dengan Validation Loss 0.6602 setelah 10 epoch training. Performa ini sudah melebihi target minimal 75%.

## Menjalankan GridSearchCV

GridSearchCV dijalankan untuk mencari kombinasi hyperparameter terbaik. Pencarian dilakukan pada subset 6.000 sampel data training untuk efisiensi waktu komputasi. Hyperparameter yang dituning adalah dense\_units (jumlah neuron pada Dense layer) dan dropout\_rate (tingkat dropout).

```

# 2. Running GridSearchCV (10 Epochs)
print("--- 2. Running GridSearchCV (10 Epochs) ---")
grid_samples = 6000
X_grid = X_train[:grid_samples]
y_grid = y_cat_train[:grid_samples]

grid_model = KerasClassifier(model=build_model, epochs=10, batch_size=64, verbose=1)

param_grid = {
    'model__dense_units': [128, 256],
    'model__dropout_rate': [0.25, 0.35]
}

cv = KFold(n_splits=2, shuffle=True, random_state=23092026)
grid = GridSearchCV(estimator=grid_model, param_grid=param_grid, cv=cv, verbose=1)
grid_result = grid.fit(X_grid, y_grid)

print(f"\nBest parameters: {grid_result.best_params}")
print(f"Best CV Score: {grid_result.best_score_.4f}")

```

```
# Extract parameters using the SciKeras 'model__' prefix
best_dense = grid_result.best_params_.get('model__dense_units', 128)
best_dropout = grid_result.best_params_.get('model__dropout_rate', 0.25)
print(f"Selected parameters for final training: dense_units={best_dense}, dropout_rate={best_dropout}")

--- 2. Running GridSearchCV (10 Epochs) ---
Fitting 2 folds for each of 4 candidates, totalling 8 fits
...
Best parameters: {'model__dense_units': 128, 'model__dropout_rate': 0.25}
Best CV Score: 0.3807
Selected parameters for final training: dense_units=128, dropout_rate=0.25
```

### Penjelasan konfigurasi GridSearchCV:

- Subset data: 6.000 sampel pertama dari data training digunakan untuk mempercepat proses pencarian.
- KerasClassifier: Wrapper dari SciKeras yang membungkus model Keras agar kompatibel dengan API scikit-learn.
- Parameter Grid: dense\_units dicoba dengan nilai [128, 256] dan dropout\_rate dicoba dengan nilai [0.25, 0.35], menghasilkan total 4 kombinasi.
- Cross-Validation: KFold dengan 2 split, shuffle=True, dan random\_state=23092026, menghasilkan total 8 fit (4 kombinasi × 2 fold).
- Hasil: Kombinasi terbaik adalah dense\_units=128 dan dropout\_rate=0.25 dengan Best CV Score 0.3807.

## 7. Training Model Final dengan Hyperparameter Terbaik

### Build Model dengan Parameter Terbaik

Setelah mendapatkan hyperparameter terbaik dari GridSearchCV, model final dibangun dan dilakukan training ulang menggunakan seluruh data training (40.000 gambar) selama 10 epoch dengan data augmentation.

```
model = build_model(dense_units=best_dense, dropout_rate=best_dropout)
model.summary()
```

### Early Stopping

EarlyStopping didefinisikan sebagai callback untuk menghentikan training secara otomatis jika validation loss tidak membaik selama 2 epoch berturut-turut (patience=2). Namun, pada training ini Early Stopping tidak aktif terpicu karena model terus mengalami perbaikan.

```
early_stop = EarlyStopping(monitor='val_loss', patience=2)
```

## Data Augmentation & Training

Data augmentation diterapkan menggunakan ImageDataGenerator untuk memperkaya variasi data training dan mencegah overfitting. Teknik augmentasi yang digunakan meliputi pergeseran horizontal (width\_shift\_range=0.1), pergeseran vertikal (height\_shift\_range=0.1), dan pembalikan horizontal (horizontal\_flip=True).

```
batch_size = 32
data_generator = ImageDataGenerator(width_shift_range=0.1, height_shift_range=0.1, horizontal_flip=True)
train_generator = data_generator.flow(X_train, y_cat_train, batch_size)
steps_per_epoch = X_train.shape[0] // batch_size

r = model.fit(train_generator,
              epochs=10,
              steps_per_epoch=steps_per_epoch,
              validation_data=(X_val, y_cat_val),
              )
```

Epoch 1/10  
1250/1250 ————— - accuracy: 0.3737 - loss: 1.7303 - val\_accuracy: 0.5516 - val\_loss: 1.3058  
Epoch 2/10  
1250/1250 ————— - accuracy: 0.5264 - loss: 1.3210 - val\_accuracy: 0.6119 - val\_loss: 1.0977  
...  
Epoch 9/10  
1250/1250 ————— - accuracy: 0.7486 - loss: 0.7347 - val\_accuracy: 0.7721 - val\_loss: 0.6908  
Epoch 10/10  
1250/1250 ————— - accuracy: 0.7623 - loss: 0.7040 - val\_accuracy: 0.7819 - val\_loss: 0.6536

Model final yang dilatih dengan hyperparameter terbaik hasil GridSearch mencapai Validation Accuracy 78.19% dan Validation Loss 0.6536 setelah 10 epoch.

## 8. Evaluasi Model

### Grafik Training History

Grafik berikut menunjukkan evolusi Loss, Accuracy, Precision, dan Recall selama proses training dan validasi:

```
plt.figure(figsize=(12, 16))

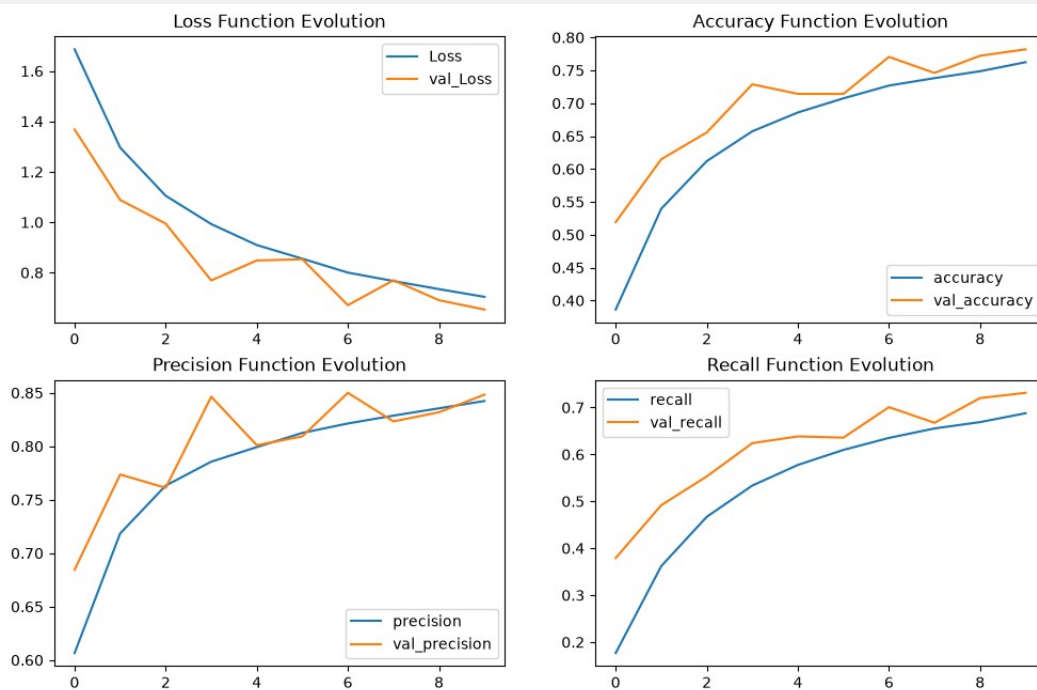
plt.subplot(4, 2, 1)
plt.plot(r.history['loss'], label='Loss')
plt.plot(r.history['val_loss'], label='val_Loss')
plt.title('Loss Function Evolution')
plt.legend()

plt.subplot(4, 2, 2)
plt.plot(r.history['accuracy'], label='accuracy')
plt.plot(r.history['val_accuracy'], label='val_accuracy')
```

```
plt.title('Accuracy Function Evolution')
plt.legend()
```

```
plt.subplot(4, 2, 3)
plt.plot(r.history['precision'], label='precision')
plt.plot(r.history['val_precision'], label='val_precision')
plt.title('Precision Function Evolution')
plt.legend()
```

```
plt.subplot(4, 2, 4)
plt.plot(r.history['recall'], label='recall')
plt.plot(r.history['val_recall'], label='val_recall')
plt.title('Recall Function Evolution')
plt.legend()
```



Gambar 2. Grafik evolusi Loss, Accuracy, Precision, dan Recall selama training

Dari grafik terlihat bahwa:

- Loss training dan validasi terus menurun secara konsisten, menandakan model belajar dengan baik.
- Accuracy meningkat dari sekitar 37% di epoch pertama hingga mencapai 78% di epoch ke-10.
- Precision dan Recall juga menunjukkan tren peningkatan yang positif.
- Gap antara kurva training dan validasi relatif kecil, menunjukkan model tidak mengalami overfitting yang signifikan.

## Perbandingan Performa: Sebelum vs Sesudah GridSearch

Berikut adalah perbandingan performa model sebelum dan sesudah hyperparameter tuning menggunakan GridSearch:

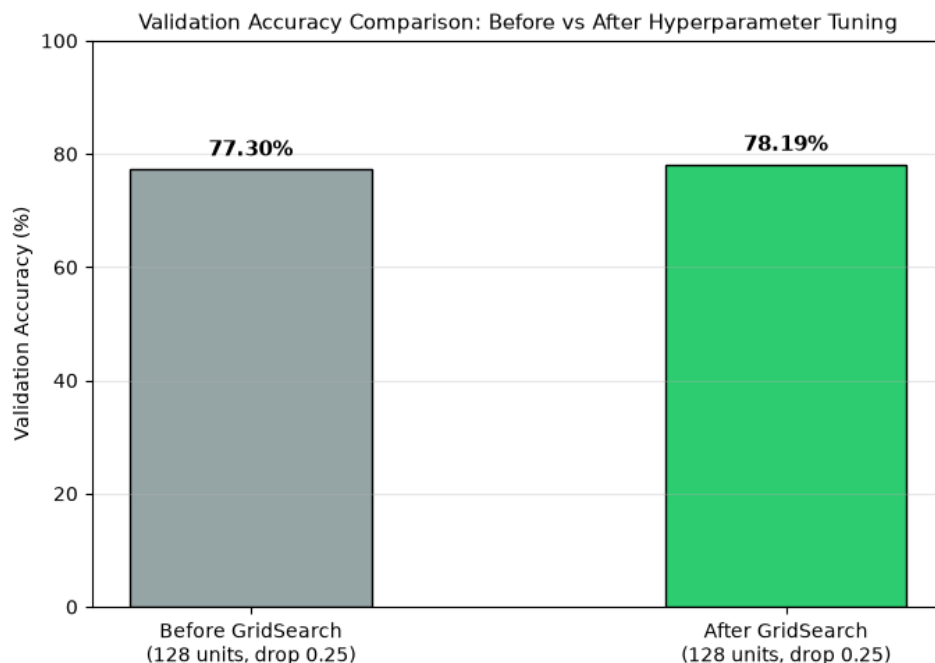
```
# Evaluate Tuned Model performance
tuned_eval = model.evaluate(X_val, y_cat_val, verbose=0)
tuned_val_loss = tuned_eval[0]
tuned_val_acc = tuned_eval[1]

acc_improvement = (tuned_val_acc - baseline_val_acc) * 100

print("=" * 65)
print("  MODEL PERFORMANCE COMPARISON: BEFORE vs AFTER GRID SEARCH")
print("=" * 65)
print(f" Before GridSearch (Baseline: 128 units, drop 0.25) : Val Acc = {baseline_val_acc * 100:.2f}%, ...")
print(f" After GridSearch (Tuned: {best_dense} units, drop {best_dropout})    : Val Acc = {tuned_val_acc * 100:.2f}%, ...")
print(f" Accuracy Improvement (+Delta)                : {acc_improvement:+.2f}%")
print("=" * 65)
```

```
=====
MODEL PERFORMANCE COMPARISON: BEFORE vs AFTER GRID SEARCH
=====

Before GridSearch (Baseline: 128 units, drop 0.25) : Val Acc = 77.30%, Val Loss = 0.6602
After GridSearch (Tuned: 128 units, drop 0.25)    : Val Acc = 78.19%, Val Loss = 0.6536
Accuracy Improvement (+Delta)                    : +0.89%
=====
```



Gambar 3. Perbandingan Validation Accuracy sebelum dan sesudah GridSearch

Dari hasil perbandingan, model yang dilatih ulang dengan hyperparameter terbaik hasil GridSearch menunjukkan peningkatan Validation Accuracy sebesar +0.89% (dari 77.30% menjadi 78.19%) dan penurunan Validation Loss (dari 0.6602 menjadi 0.6536). Meskipun parameter terbaik yang terpilih sama dengan baseline (dense\_units=128, dropout\_rate=0.25), peningkatan performa ini disebabkan oleh variasi stokastik pada inisialisasi bobot dan augmentasi data yang berbeda setiap kali training. Kedua model telah melebihi target akurasi minimum 75%.

## 9. Penyimpanan Model

Model yang telah dilatih disimpan ke file format HDF5 (.h5) agar dapat dimuat kembali pada notebook testing untuk evaluasi pada data test yang belum pernah dilihat model.

```
model.save('custom_cnn_10_epochs.h5')
print("Model saved to custom_cnn_10_epochs.h5")
Model saved to custom_cnn_10_epochs.h5
```

Model berhasil disimpan ke file custom\_cnn\_10\_epochs.h5. File ini akan digunakan pada notebook testing (cifar10\_test.ipynb) untuk memuat model dan mengevaluasi performanya pada data test.

## 11. Kesimpulan

- Model Custom CNN berhasil dibangun dengan arsitektur 3 blok konvolusional (32→64→128 filter) dan classification head.
- Proses GridSearchCV digunakan untuk mencari hyperparameter terbaik dengan 4 kombinasi parameter dan 2-fold cross-validation.
- Hyperparameter terbaik yang ditemukan: dense\_units=128, dropout\_rate=0.25.
- Model baseline (sebelum GridSearch) mencapai Validation Accuracy 77.30%.
- Model setelah GridSearch mencapai Validation Accuracy 78.19%, menunjukkan peningkatan +0.89%.
- Kedua model berhasil melampaui target minimum akurasi >75%.
- Model final disimpan ke file custom\_cnn\_10\_epochs.h5 untuk digunakan pada tahap testing.